Day 3: Object-Oriented Programming

Java OOP Practice Programs & Console Student Management Application

Topic	Covered
Classes and objects	Student, Employee, Manager
Constructors	Default, parameterized, chaining
Encapsulation	Private fields, validated getters/setters
Inheritance	Single and multilevel
Polymorphism	Overriding, Person[] reference array
Abstraction	Abstract class and interface
Access modifiers	private, protected, default, public
Method overloading	addMarks(), calculateBonus()

1. Core Concepts

Classes and Objects

A **class** is a blueprint that defines properties (fields) and behaviour (methods). An **object** is an instance of a class created with `new`. Example: `Student s1 = new Student(101, "Aarav", 20);`

Constructors

A special method with the same name as the class, run when an object is created. Constructors can be overloaded, and `this(...)` or `super(...)` chains to another constructor in the same or parent class.

Encapsulation

Hiding data by making fields `private` and exposing them through public getters and setters. Setters can validate input, so objects never end up in an invalid state.

Inheritance

A subclass acquires the fields and methods of its parent using `extends`. This promotes code reuse and models an IS-A relationship (Student IS-A Person).

Polymorphism

One interface, many forms. **Compile-time**: method overloading. **Runtime**: method overriding, where a parent reference calls the child's version of a method.

Abstraction

Showing only essential details. Achieved with `abstract` classes (cannot be instantiated; may hold abstract methods) and `interfaces` (a pure contract).

Method Overloading

Several methods share a name but differ in parameter list (number, type or order). The compiler picks the right one from the arguments.

Access Modifiers

Modifier	Same class	Same package	Subclass (other pkg)	Everywhere
private	Yes	No	No	No
default (none)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Class Hierarchy

```
Printable (interface)
    |
Person (abstract)  - name, age (private) | getRole() abstract
    |-- Student    - rollNo, marks | addMarks() overloaded
    |-- Employee   - empId, salary | calculateBonus() overloaded
            |-- Manager - teamSize | calculateBonus() overridden
```

2. OOP Practice Program: OOPDemo.java

```
/**
 * Day 3 - OOP Practice Program
 * Covers: classes/objects, constructors, encapsulation, inheritance,
 * polymorphism, abstraction, access modifiers, method overloading.
 */

// ABSTRACTION: interface defines WHAT, not HOW
interface Printable {
    void printDetails();
}

// ABSTRACTION: abstract class cannot be instantiated
abstract class Person implements Printable {
    // ENCAPSULATION: private fields, accessed only via getters/setters
    private String name;
    private int age;
    protected String country = "India";   // protected: visible to subclasses
    String category = "General";          // default (package-private)
    public static int totalPeople = 0;    // public static: shared by all objects

    // CONSTRUCTORS (overloaded)
    public Person() {
        this("Unknown", 18);              // constructor chaining
    }

    public Person(String name, int age) {
        setName(name);
        setAge(age);
        totalPeople++;
    }

    // Getters & setters with validation
    public String getName() { return name; }
    public void setName(String name) {
        if (name == null || name.trim().isEmpty()) {
            System.out.println("Invalid name, keeping previous value.");
            return;
        }
        this.name = name.trim();
    }

    public int getAge() { return age; }
    public void setAge(int age) {
        if (age < 1 || age > 120) {
            System.out.println("Invalid age " + age + ", defaulting to 18.");
            this.age = 18;
        } else {
            this.age = age;
        }
    }

    // Abstract method: every subclass MUST implement
    public abstract String getRole();

    @Override
    public void printDetails() {
        System.out.println(getRole() + " | Name: " + name + " | Age: " + age + " | Country: " + country);
    }
}

// INHERITANCE: Student IS-A Person
class Student extends Person {

    private int rollNo;
    private double[] marks;

    public Student(int rollNo, String name, int age) {
        super(name, age);                 // call parent constructor
        this.rollNo = rollNo;
        this.marks = new double[0];
    }

    public int getRollNo() { return rollNo; }

    // METHOD OVERLOADING: same name, different parameters
    public void addMarks(double mark) {
        double[] copy = new double[marks.length + 1];
        System.arraycopy(marks, 0, copy, 0, marks.length);
        copy[marks.length] = mark;
        marks = copy;
```

```
    }

    public void addMarks(double... newMarks) {   // varargs version
        for (double m : newMarks) addMarks(m);
    }

    public double getAverage() {
        if (marks.length == 0) return 0;
        double sum = 0;
        for (double m : marks) sum += m;
        return sum / marks.length;
    }

    public String getGrade() {
        double avg = getAverage();
        if (avg >= 90) return "A+";
        if (avg >= 80) return "A";
        if (avg >= 70) return "B";
        if (avg >= 60) return "C";
        if (avg >= 40) return "D";
        return "F";
    }

    @Override
    public String getRole() { return "Student"; }

    // METHOD OVERRIDING (runtime polymorphism)
    @Override
    public void printDetails() {
        super.printDetails();
        System.out.printf("   Roll No: %d | Average: %.2f | Grade: %s%n", rollNo, getAverage(), getGrade());
    }
}

// INHERITANCE: Employee IS-A Person
class Employee extends Person {
    private int empId;
    private double salary;

    public Employee(int empId, String name, int age, double salary) {
        super(name, age);
        this.empId = empId;
        setSalary(salary);
    }

    public int getEmpId() { return empId; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) {
        if (salary < 0) {
            System.out.println("Salary cannot be negative.");
            return;
        }
        this.salary = salary;
    }

    // METHOD OVERLOADING
    public double calculateBonus() { return salary * 0.10; }                  // default 10%
    public double calculateBonus(double percent) { return salary * percent / 100; }
    public double calculateBonus(double percent, double fixedExtra) {
        return salary * percent / 100 + fixedExtra;
    }

    @Override
    public String getRole() { return "Employee"; }

    @Override
    public void printDetails() {
        super.printDetails();
        System.out.printf("   Emp ID: %d | Salary: %.2f | Default Bonus: %.2f%n", empId, salary, calculateBonus());
    }
}

// MULTILEVEL INHERITANCE: Manager -> Employee -> Person
class Manager extends Employee {
    private int teamSize;

    public Manager(int empId, String name, int age, double salary, int teamSize) {
        super(empId, name, age, salary);
        this.teamSize = teamSize;
    }

    @Override
    public String getRole() { return "Manager"; }
```

```
    @Override
    public double calculateBonus() {          // overriding: managers get 20% + team bonus
        return getSalary() * 0.20 + teamSize * 1000;
    }

    @Override
    public void printDetails() {
        super.printDetails();
        System.out.println("   Team Size: " + teamSize);
    }
}

public class OOPDemo {
    public static void main(String[] args) {
        System.out.println("===== 1. Classes, Objects & Constructors =====");
        Student s1 = new Student(101, "Aarav", 20);
        Student s2 = new Student(102, "Diya", 19);
        s1.addMarks(85);                        // single value overload
        s1.addMarks(90, 78, 92);                // varargs overload
        s2.addMarks(65, 70, 58);
        s1.printDetails();
        s2.printDetails();

        System.out.println("\n===== 2. Encapsulation (validation) =====");
        s1.setAge(-5);                          // rejected -> 18
        s1.setName("   ");                      // rejected
        System.out.println("Name still: " + s1.getName() + ", Age: " + s1.getAge());

        System.out.println("\n===== 3. Inheritance & Method Overloading =====");
        Employee e1 = new Employee(5001, "Rahul", 30, 50000);
        Manager m1 = new Manager(5002, "Sneha", 38, 90000, 8);
        e1.printDetails();
        m1.printDetails();
        System.out.println("Bonus (default): " + e1.calculateBonus());
        System.out.println("Bonus (15%):     " + e1.calculateBonus(15));
        System.out.println("Bonus (15%+500): " + e1.calculateBonus(15, 500));

        System.out.println("\n===== 4. Polymorphism & Abstraction =====");
        Person[] people = { s1, s2, e1, m1 };  // Person reference, different objects
        for (Person p : people) {
            p.printDetails();                   // correct overridden method runs at runtime
        }
        // Person p = new Person();             // ERROR: abstract class can't be instantiated

        System.out.println("\n===== 5. Static member & instanceof =====");
        System.out.println("Total people created: " + Person.totalPeople);
        for (Person p : people) {
            if (p instanceof Manager) System.out.println(p.getName() + " is a Manager");
            else if (p instanceof Employee) System.out.println(p.getName() + " is an Employee");
            else System.out.println(p.getName() + " is a Student");
        }
    }
}
```

Output

```
===== 1. Classes, Objects & Constructors =====
Student | Name: Aarav | Age: 20 | Country: India
   Roll No: 101 | Average: 86.25 | Grade: A
Student | Name: Diya | Age: 19 | Country: India
   Roll No: 102 | Average: 64.33 | Grade: C

===== 2. Encapsulation (validation) =====
Invalid age -5, defaulting to 18.
Invalid name, keeping previous value.
Name still: Aarav, Age: 18

===== 3. Inheritance & Method Overloading =====
Employee | Name: Rahul | Age: 30 | Country: India
   Emp ID: 5001 | Salary: 50000.00 | Default Bonus: 5000.00
Manager | Name: Sneha | Age: 38 | Country: India
   Emp ID: 5002 | Salary: 90000.00 | Default Bonus: 26000.00
   Team Size: 8
Bonus (default): 5000.0
Bonus (15%):     7500.0
Bonus (15%+500): 8000.0

===== 4. Polymorphism & Abstraction =====
Student | Name: Aarav | Age: 18 | Country: India
   Roll No: 101 | Average: 86.25 | Grade: A
Student | Name: Diya | Age: 19 | Country: India
   Roll No: 102 | Average: 64.33 | Grade: C
Employee | Name: Rahul | Age: 30 | Country: India
```

```
   Emp ID: 5001 | Salary: 50000.00 | Default Bonus: 5000.00
Manager | Name: Sneha | Age: 38 | Country: India
   Emp ID: 5002 | Salary: 90000.00 | Default Bonus: 26000.00
   Team Size: 8

===== 5. Static member & instanceof =====
Total people created: 4
Aarav is a Student
Diya is a Student
Rahul is an Employee
Sneha is a Manager
```

3. Console Student Management Application

A menu-driven console app that lets you add, view, search (by ID or name), update and delete student records, and see statistics. It is split into three classes: **StudentRecord** (encapsulated data), **StudentManager** (CRUD logic over an ArrayList) and **StudentManagementApp** (menu and validated input).

Menu option	Action
1	Add student (rejects duplicate IDs, validates marks 0-100)
2	View all students in a formatted table
3 / 4	Search by ID / search by name keyword
5	Update name, course or marks (blank keeps current value)
6	Delete a student by ID
7	Statistics: total, class average, topper
0	Exit

Source: StudentManagementApp.java

```
import java.util.ArrayList;
import java.util.Scanner;

/** A simple student record with encapsulated fields. */
class StudentRecord {
    private final int id;
    private String name;
    private int age;
    private String course;
    private double marks;

    public StudentRecord(int id, String name, int age, String course, double marks) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.course = course;
        this.marks = marks;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getCourse() { return course; }
    public double getMarks() { return marks; }

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setCourse(String course) { this.course = course; }
    public void setMarks(double marks) { this.marks = marks; }

    public String getGrade() {
        if (marks >= 90) return "A+";
        if (marks >= 80) return "A";
        if (marks >= 70) return "B";
        if (marks >= 60) return "C";
        if (marks >= 40) return "D";
        return "F";
    }

    @Override
    public String toString() {
        return String.format("| %-4d | %-15s | %-3d | %-12s | %6.2f | %-5s |",
                id, name, age, course, marks, getGrade());
    }
}

/** Handles the collection of students (CRUD operations). */
class StudentManager {
    private final ArrayList<StudentRecord> students = new ArrayList<>();

    public boolean add(StudentRecord s) {
```

```
        if (find(s.getId()) != null) return false;   // no duplicate IDs
        students.add(s);
        return true;
    }

    public StudentRecord find(int id) {
        for (StudentRecord s : students) {
            if (s.getId() == id) return s;
        }
        return null;
    }

    public boolean remove(int id) {
        StudentRecord s = find(id);
        return s != null && students.remove(s);
    }

    public ArrayList<StudentRecord> searchByName(String keyword) {
        ArrayList<StudentRecord> result = new ArrayList<>();
        for (StudentRecord s : students) {
            if (s.getName().toLowerCase().contains(keyword.toLowerCase())) result.add(s);
        }
        return result;
    }

    public StudentRecord topper() {
        StudentRecord best = null;
        for (StudentRecord s : students) {
            if (best == null || s.getMarks() > best.getMarks()) best = s;
        }
        return best;
    }

    public double average() {
        if (students.isEmpty()) return 0;
        double sum = 0;
        for (StudentRecord s : students) sum += s.getMarks();
        return sum / students.size();
    }

    public ArrayList<StudentRecord> all() { return students; }
    public int count() { return students.size(); }
}

public class StudentManagementApp {
    private static final Scanner sc = new Scanner(System.in);
    private static final StudentManager manager = new StudentManager();

    public static void main(String[] args) {
        // sample data so the app isn't empty on first run
        manager.add(new StudentRecord(1, "Aarav Sharma", 20, "Computer", 88.5));
        manager.add(new StudentRecord(2, "Diya Patil", 19, "Maths", 72.0));

        int choice;
        do {
            printMenu();
            choice = readInt("Enter choice: ");
            switch (choice) {
                case 1: addStudent(); break;
                case 2: viewAll(); break;
                case 3: searchById(); break;
                case 4: searchByName(); break;
                case 5: updateStudent(); break;
                case 6: deleteStudent(); break;
                case 7: showStats(); break;
                case 0: System.out.println("Goodbye!"); break;
                default: System.out.println("Invalid choice, try again.");
            }
        } while (choice != 0);
    }

    private static void printMenu() {
        System.out.println("\n===== STUDENT MANAGEMENT SYSTEM =====");

        System.out.println("1. Add Student");
        System.out.println("2. View All Students");
        System.out.println("3. Search by ID");
        System.out.println("4. Search by Name");
        System.out.println("5. Update Student");
        System.out.println("6. Delete Student");
        System.out.println("7. Statistics (Topper & Average)");
        System.out.println("0. Exit");
    }
```

```
private static void addStudent() {
    int id = readInt("ID: ");
    if (manager.find(id) != null) {
        System.out.println("A student with this ID already exists.");
        return;
    }
    String name = readString("Name: ");
    int age = readInt("Age: ");
    String course = readString("Course: ");
    double marks = readMarks();
    manager.add(new StudentRecord(id, name, age, course, marks));
    System.out.println("Student added successfully.");
}

private static void viewAll() {
    if (manager.count() == 0) {
        System.out.println("No records found.");
        return;
    }
    printHeader();
    for (StudentRecord s : manager.all()) System.out.println(s);
    System.out.println("+------+-----------------+-----+-------------+--------+-------+");
}

private static void searchById() {
    StudentRecord s = manager.find(readInt("Enter ID: "));
    if (s == null) { System.out.println("Student not found."); return; }
    printHeader();
    System.out.println(s);
}

private static void searchByName() {
    ArrayList<StudentRecord> found = manager.searchByName(readString("Enter name/keyword: "));
    if (found.isEmpty()) { System.out.println("No matches."); return; }
    printHeader();
    for (StudentRecord s : found) System.out.println(s);
}

private static void updateStudent() {
    StudentRecord s = manager.find(readInt("Enter ID to update: "));
    if (s == null) { System.out.println("Student not found."); return; }
    System.out.println("Leave blank to keep the current value.");
    String name = readString("New name [" + s.getName() + "]: ");
    if (!name.isEmpty()) s.setName(name);
    String course = readString("New course [" + s.getCourse() + "]: ");
    if (!course.isEmpty()) s.setCourse(course);
    String marks = readString("New marks [" + s.getMarks() + "]: ");
    if (!marks.isEmpty()) {
        try {
            double m = Double.parseDouble(marks);
            if (m >= 0 && m <= 100) s.setMarks(m);
            else System.out.println("Marks must be 0-100; unchanged.");

        } catch (NumberFormatException e) {
            System.out.println("Invalid number; marks unchanged.");
        }
    }
    System.out.println("Student updated.");
}

private static void deleteStudent() {
    int id = readInt("Enter ID to delete: ");
    System.out.println(manager.remove(id) ? "Student deleted." : "Student not found.");
}

private static void showStats() {
    if (manager.count() == 0) { System.out.println("No records found."); return; }
    StudentRecord t = manager.topper();
    System.out.println("Total students : " + manager.count());
    System.out.printf("Class average  : %.2f%n", manager.average());
    System.out.println("Topper         : " + t.getName() + " (" + t.getMarks() + ")");
}

// ---------- input helpers with validation ----------
private static void printHeader() {
    System.out.println("+------+-----------------+-----+-------------+--------+-------+");
    System.out.println("| ID   | Name            | Age | Course      | Marks  | Grade |");
    System.out.println("+------+-----------------+-----+-------------+--------+-------+");
}

private static int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
```

```
            try {
                return Integer.parseInt(sc.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid whole number.");
            }
        }
    }

    private static double readMarks() {
        while (true) {
            System.out.print("Marks (0-100): ");
            try {
                double m = Double.parseDouble(sc.nextLine().trim());
                if (m >= 0 && m <= 100) return m;
                System.out.println("Marks must be between 0 and 100.");
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid number.");
            }
        }
    }

    private static String readString(String prompt) {
        System.out.print(prompt);
        return sc.nextLine().trim();
    }
}
```

Sample Session

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View All Students
3. Search by ID
4. Search by Name
5. Update Student
6. Delete Student
7. Statistics (Topper & Average)
0. Exit
Enter choice: +------+-----------------+-----+--------------+--------+-------+
| ID   | Name            | Age | Course       | Marks  | Grade |
+------+-----------------+-----+--------------+--------+-------+
| 1    | Aarav Sharma    | 20  | Computer     |  88.50 | A     |
| 2    | Diya Patil      | 19  | Maths        |  72.00 | B     |
+------+-----------------+-----+--------------+--------+-------+

[ menu displayed again ]
Enter choice: Enter ID: +------+-----------------+-----+--------------+--------+-------+
| ID   | Name            | Age | Course       | Marks  | Grade |
+------+-----------------+-----+--------------+--------+-------+
| 1    | Aarav Sharma    | 20  | Computer     |  88.50 | A     |

[ menu displayed again ]
Enter choice: Enter name/keyword: +------+-----------------+-----+--------------+--------+-------+
| ID   | Name            | Age | Course       | Marks  | Grade |
+------+-----------------+-----+--------------+--------+-------+
| 2    | Diya Patil      | 19  | Maths        |  72.00 | B     |

[ menu displayed again ]
Enter choice: Total students : 2
Class average  : 80.25
Topper         : Aarav Sharma (88.5)

[ menu displayed again ]
Enter choice: ID: Name: Age: Course: Marks (0-100): Student added successfully.

[ menu displayed again ]
Enter choice: +------+-----------------+-----+--------------+--------+-------+
| ID   | Name            | Age | Course       | Marks  | Grade |
+------+-----------------+-----+--------------+--------+-------+
| 1    | Aarav Sharma    | 20  | Computer     |  88.50 | A     |
| 2    | Diya Patil      | 19  | Maths        |  72.00 | B     |
| 3    | Rohan           | 21  | CS           |  95.00 | A+    |
+------+-----------------+-----+--------------+--------+-------+

[ menu displayed again ]
Enter choice: Total students : 3
Class average  : 85.17
Topper         : Rohan (95.0)

[ menu displayed again ]
Enter choice: Enter ID to delete: Student deleted.

[ menu displayed again ]
```

```
Enter choice: Goodbye!
```

How to Run

```
javac OOPDemo.java StudentManagementApp.java
java OOPDemo
java StudentManagementApp
```